

Module A — Environnement (VM RE-Lab)

La même VM Linux Vagrant sert tout le cours (A → E). Elle est **headless** : on travaille en **ligne de commande** (SSH). Les outils graphiques (Ghidra, Burp) sont **optionnels** et réservés aux bonus.

Démarrage rapide

Prérequis sur l'hôte : **Vagrant**, plus **un** des deux hyperviseurs — **VirtualBox** (défaut) ou **VMware Workstation / Fusion**.

```
cd vagrant/  
vagrant up                # crée + provisionne (~10-15 min la 1re fois)  
vagrant ssh               # connexion  
~/labs/check_tools.sh     # vérifie que tout est là
```

Avec VMware plutôt que VirtualBox

Le **Vagrantfile** déclare les deux hyperviseurs : même box, même provisioning, mêmes TP. Seule la commande de démarrage change.

Installation, une seule fois, **deux composants** (le plugin seul ne suffit pas) :

```
vagrant plugin install vagrant-vmware-desktop  
# puis installer le « Vagrant VMware Utility » (démon système, paquet  
séparé  
# à télécharger chez HashiCorp :  
developer.hashicorp.com/vagrant/install/vmware)
```

Puis :

```
vagrant up --provider=vmware_desktop
```

Pour ne pas le retaper à chaque fois : **export VAGRANT_DEFAULT_PROVIDER=vmware_desktop**.

Deux points d'attention côté VMware :

- Le **Vagrant VMware Utility** doit tourner (c'est lui qui crée le réseau host-only **192.168.56.0/24** exigé par les TP malware). S'il est absent, le **vagrant up** échoue sur l'étape réseau.
- Le dossier partagé passe par **HGFS** (open-vm-tools) au lieu de vboxsf. Aucune conséquence sur les TP : **provision.sh** recopie les cibles sur le disque natif de la VM avant usage.

Ne pas mélanger les deux : une VM créée sous un hyperviseur n'est pas réutilisable sous l'autre. Pour changer, faire **vagrant destroy** puis relancer avec l'autre provider.

Avant tout TP manipulant du code malveillant :

```
vagrant snapshot save clean      # depuis l'hôte, VM éteinte ou allumée
# ... TP ...
vagrant snapshot restore clean   # remet la VM propre
```

Ce qui est installé (headless / CLI)

Besoin	Outils
RE binaire natif	radare2/r2, objdump, readelf, nm, file, strings, xxd
Débogage	gdb + pwndbg, ltrace, strace
Exploitation / scripting	python3, pwntools, capstone, ROPgadget
Décompilation .NET / C#	ilspycmd (.NET SDK 8, cross-platform)
WebAssembly	wasm2wat, wasm-decompile (WABT)
Analyse de malware	yara, strings, file, radare2, réseau host-only
Web	node, npm (dé-obfuscation JS en CLI)

randomize_va_space (ASLR) est laissé par défaut ; on le désactive **par TP** quand c'est pédagogiquement nécessaire (Module C), jamais en global.

Réseau & isolation

- La VM est en **private_network host-only (192.168.56.20)** : **pas d'accès Internet** pour les binaires analysés.
- Tout TP malware/ransomware se fait **snapshot clean pris au préalable**, dans `~/labs/A_re-malware/lab-sandbox/` uniquement.
- Règle d'or : **on n'exécute un binaire suspect que si on a compris ce qu'il fait** ; sinon, analyse **statique** d'abord.

Cibles du Module A

Les cibles sont **bénignes** et fournies dans `~/labs/cibles/` (déposées via le dossier `vagrant/shared/`). Voir `A_TP_enonces.md` :

- A1** — **licensecheck** : petite appli **.NET single-file** (crackme de licence). *Maison* — spec dans le corrigé.
- A2** — **fakestealer** : binaire **bénin** qui *simule* des IoC (dépose un fichier marqueur, tente une connexion vers `192.168.56.1`, chaînes suspectes) sans aucune charge réelle. *Maison*.
- A3** — **toycrypt** : démonstrateur **bénin** de chiffrement, agissant **uniquement** sur les fichiers factices de `lab-sandbox/`, clé récupérable. *Maison* — sert de fil rouge vers les modules C et E.

Si tu préfères des cibles publiques toutes prêtes : crackmes.one (A1), et pour A2/A3 des **échantillons pédagogiques bénins** de dépôts de cours (jamais de vraie souche in-the-wild).

Dépannage

- **pwndbg absent** : `cd /opt/pwndbg && ./setup.sh` puis vérifier `~/.gdbinit`.
- **ilspycmd introuvable** : `source ~/.bashrc` (le `PATH ~/.dotnet/tools` est ajouté au provisioning).
- **wasn2wat absent** (apt trop vieux) : récupérer la release WABT depuis GitHub et l'ajouter au PATH.
- **VM lente** : réduire `vb.memory/vb.cpus` dans le `Vagrantfile`, ou fermer les autres VM.

Bonus GUI (optionnel, hors tronc commun)

Pour ceux qui veulent Ghidra en fenêtre : installer un bureau léger (`xfce4` + serveur X ou X-forwarding `vagrant ssh -- -X`) puis Ghidra. **Non requis** : tout le tronc commun se fait en CLI (`radare2`, `gdb`, `ilspycmd`).